Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 1

The OS as a Resource Manager

Track A: The Real OS, from the Outside (Linux)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1 Introduction **2**

2 System Information **2**

3 Files and Directories **2**

3.1 Observations 3

3.2 Interesting Command Output 3

4 Processes **3**

4.1 Observations 4

4.2 Interesting Command Output 4

5 Memory Management **5**

5.1 Observations 5

5.2 Interesting Command Output 5

6 Devices and Storage **6**

6.1 Observations 6

6.2 Interesting Command Output 6

7 Conclusion **7**

1. Introduction

The purpose of this laboratory work is to observe how an operating system manages the main resources of a computer system. The experiment was performed in Ubuntu running inside an Oracle VirtualBox virtual machine. During the laboratory work, four major areas of operating system resource management were investigated: files and directories, processes, memory, and devices/storage.

This was my first practical work with Linux commands, so the main objective was also to become familiar with the terminal and understand what information the operating system returns after each command.

2. System Information

The following commands were used to identify the current user, Linux kernel, and system uptime:

```
whoami
uname -a
uptime
```

The system reported:

```
alex_007m
Linux Mashinka 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC
Wed Sep 2 14:29:37 UTC 2026 x86_64 GNU/Linux
14:45:17 up 4 min, 1 user, load average: 0.03, 0.10, 0.05
```

The current user was `alex_007m`. The machine was running a 64-bit Linux kernel, and at the moment of the test the virtual machine had been running for about four minutes.

3. Files and Directories

In the first part of the laboratory work, I explored the Linux file system and performed basic file-management operations.

The current directory and main Linux directories were inspected using:

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
```

A demonstration directory and files were then created and modified:

```
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
```

```
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

3.1. Observations

1. The files created during the experiment were owned by user `alex_007m` and belonged to group `alex_007m`.
2. After executing

   ```
   chmod 600 note.txt
   ```

   the file permissions became:

   ```
   -rw-------
   ```

   The first character, `-`, indicates that `note.txt` is a regular file. The next three characters, `rw-`, mean that the owner can read and write the file but cannot execute it. The next three characters mean that the group has no permissions, and the last three mean that other users also have no permissions.
3. Two directories directly under the root directory `/` are:
   - `/etc` – contains system-wide configuration files.
   - `/home` – contains users' home directories.

3.2. Interesting Command Output

The command `ls -l` displayed:

```
total 8
-rw-rw-r-- 1 alex_007m alex_007m 24 Sep 28 14:50 note.txt
-rw-rw-r-- 1 alex_007m alex_007m 24 Sep 28 14:50 renamed.txt
```

This output demonstrates that Linux keeps information about file type, permissions, owner, group, size, and modification time for every file.

4. Processes

The second part of the laboratory work focused on process management. The currently running processes were inspected using:

```
ps aux | head
ps aux | wc -l
top
```

A new background process was created with:

```
sleep 300 &
```

The process was inspected using:

```
jobs
ps -ef | grep sleep
```

The first test process had PID `4388` and was terminated with:

```
kill 4388
```

The Linux pseudo-filesystem `/proc` was also used to inspect information maintained by the kernel about another `sleep` process:

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
```

4.1. Observations

1. Process number 1 had PID 1 and was `systemd`. It is the main system process responsible for starting and managing many system services.
2. Approximately 210 process entries were present while the virtual machine was mostly idle.
3. For the test `sleep` process, the status file contained:

   ```
   State: S (sleeping)
   ```

   This means the process existed but was waiting instead of actively using the CPU.

4.2. Interesting Command Output

The process status contained:

```
Name:    sleep
State:   S (sleeping)
Tgid:    4427
Pid:     4427
PPid:    4099
VmPeak:  16112 kB
VmSize:  16112 kB
```

This showed me that the operating system stores detailed information about every process and exposes part of it through the `/proc` filesystem.

5. Memory Management

The third part of the laboratory work investigated how Linux manages physical memory and swap space.

The following commands were executed:

```
free -h
cat /proc/meminfo | head -6
```

The resident memory used by a simple `sleep` process was inspected with:

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

5.1. Observations

1. The virtual machine had approximately **3.3 GiB** of total RAM. At the time of observation, about **724 MiB** was completely free, and about **2.1 GiB** was available to applications.

2. Swap is disk space that the operating system can use as an extension of physical memory when RAM is under pressure. In this virtual machine, no swap was configured:

   ```
   Swap: 0B  0B  0B
   ```

3. The `sleep` process used approximately **7720 kB** of resident memory according to its `VmRSS` value. I expected a simple waiting process to use very little memory, so it was interesting to see that it still requires several megabytes of RAM.

5.2. Interesting Command Output

```
               total        used        free      shared  buff/cache   available
Mem:           3.3Gi       1.2Gi       724Mi        24Mi       1.6Gi       2.1Gi
Swap:             0B          0B          0B
```

The output demonstrates how Linux divides physical memory between currently used memory, completely free memory, cached data, and memory that is still available to programs.

6. Devices and Storage

The final experimental part focused on storage devices, mounted filesystems, and device interfaces.

The following commands were used:

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

The disk-space information showed the root filesystem as:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        30G  6.6G   22G  24% /
```

6.1. Observations

1. The root filesystem `/` was mounted on:

   ```
   /dev/sda2
   ```

2. One example of a device interface is `/dev/sda`. It represents the virtual hard disk connected to the virtual machine. Its partition `/dev/sda2` contains the Linux root filesystem.

3. The UNIX principle "everything is a file" means that many operating-system resources, including devices and kernel interfaces, are represented using file-like paths that programs can access with common operations.

6.2. Interesting Command Output

The `lsblk` command showed the block devices visible to Linux:

```
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0    30G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    30G  0 part /
sr0     11:0    1  1024M  1 rom
```

The output clearly shows the virtual disk `sda`, its partitions, and that `sda2` is mounted as the root filesystem. The working folder itself occupied only about `12K`.

7. Conclusion

The operating system manages four major categories of resources: files, processes, memory, and devices/storage.

File management was observed using commands such as `ls` and `chmod`; process management was inspected using `ps`, `top`, and `/proc`; memory usage was examined using `free -h`; and storage devices were inspected using `df -h` and `lsblk`.

This laboratory work helped me understand that the operating system is not only the graphical interface. Linux continuously manages files, running programs, RAM, and hardware resources in the background, and the terminal provides a direct way to observe this work.